

송태현

세 개의 프로젝트로 증명한 문제 해결과 성장 —
Java/Spring 으로 안전한 업무 흐름을 설계하는 백엔드 개발자

Java 21

Spring Boot 3.5

Spring Security · JWT

JPA · MyBatis

PostgreSQL

WebSocket · SSE

EC2 · nginx · GitHub Actions

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502

3 PROJECTS · 2026.05 – 09
Cooking Star · Haru Health · 땡땡땡

목차 — 프로젝트별로 독립적으로 읽을 수 있게 구성했습니다

각 프로젝트는 담당 범위 → 아키텍처 → 문제 해결 사례 → 검증 · 한계 순서입니다. 제목이나 페이지를 누르면 해당 장으로 이동하고, 모든 장 오른쪽 아래의 '🔄 목차'로 돌아옵니다.

01

[소개](#) · [기술 스택](#) · [세 프로젝트 한눈에](#)

일하는 방식 · 어디에 왜 썼는지 · 프로젝트 간 개선 흐름

[03 - 05](#) →

02

[댕댕댕 — 인증](#) · [채팅](#) · [공통 규약](#) · [배포 \(2026.08-09\)](#)

토큰 재사용 감지 롤백 버그 · REST+STOMP · AFTER_COMMIT · 커넥션 풀 5 개 · EC2 배포

[06 - 10](#) →

03

[Haru Health — 결제](#) · [정산](#) · [출석](#) · [알림 \(2026.06-07\)](#)

결제 원자성 · PT 차감 원장 · advisory lock · LATERAL · SSE 티켓

[11 - 14](#) →

04

[Cooking Star — 회원](#) · [게시판](#) · [외부 API \(2026.05\)](#)

첫 프로젝트의 기본기와 한계 → 다음 프로젝트의 기준

[15 - 16](#) →

05

[성장](#) · [한계](#) · [현재 보완 과제](#)

확인한 것과 부족한 것을 나란히

[17 - 18](#) →

시간이 3 분뿐이라면

이 네 장만 보셔도 됩니다. 제목을 누르면 이동합니다.

05

[세 프로젝트 한눈에](#)

규모 · 담당 · 한계를 한 장에

08

[댕댕댕 · 트랜잭션 롤백 버그](#)

재현 → 대안 → 수정 → 결과

13

[Haru · 결제 트랜잭션 · PT 원장](#)

행위자 / 대상 분리 · 조건부 UPDATE

17

[성장 · 한계 · 현재 보완 과제](#)

검증 범위와 과제

송태현 — 권한 · 트랜잭션 · 데이터 정합성 · 실시간 처리를 설계하는 백엔드 개발자

경영학을 공부하다 개발로 전향했습니다. '틀리면 안 되는 흐름'을 안전하게 만드는 일에 집중합니다.

송태현

Backend Developer · 2001

학력

협성대학교 경영학과

교육

실무로 배우는 생성형 AI & 클라우드 서비스 개발
전문가 과정 (GPT & LLM 활용) · 6 개월 · 2026

지향

백엔드 전문 개발자 — Java/Spring · PostgreSQL
기반 업무 시스템

연락

uiop7000@naver.com
github.com/Taehyun-0502

일하는 방식

1

권한과 입력은 서버가 판단한다

Cooking Star — Principal·Role·SQL 작성자 조건으로 소유권 확인

2

함께 성공할 일과 실패해도 되는 일을 나눈다

Haru Health — 결제 원자성 · 알림 예외 전파 제한 · 도메인 원장 분리

3

제약을 설계 조건으로 바꾼다

댕댕댕 — 커넥션 풀 5 개 · 토큰 회전 · 커밋 후 방송을 설계에 반영

2026 성장 타임라인

05



Cooking Star

기본기 · 2 인

06-07



Haru Health

업무 정합성 · 4 인

08-09



댕댕댕

운영 설계 · 배포 · 4 인

현재



보완 과제

DB 검증 · CI

기술 이름보다 '어디에 왜 썼는지' — 세 프로젝트에서 실제 연결한 역할만 적었습니다

● = 세 프로젝트 공통 스택 . 기술별 사용 맥락과 실제 담당 범위를 함께 표시했습니다 .

BACKEND

● Java 21

record · Stream · Optional 중심 서비스 계층

● Spring Boot 3.5

MVC · @Transactional 경계 · 스케줄러 · 이벤트

● Spring Security

FormLogin → JWT 필터 체인 (OncePerRequestFilter)

Spring Data JPA

댕댕댕 — @Version · 도메인 이벤트 · AFTER_COMMIT

MyBatis

Haru · 쿼리스타 — 동적 SQL · <include> 조각

DATABASE

● PostgreSQL

LATERAL · DISTINCT ON · FILTER · ON CONFLICT

Supabase (풀러)

트랜잭션 모드 · pool 5 제약을 설계 조건으로

HikariCP

prepareThreshold=0 트러블슈팅

제약 설계

부분 유니크 인덱스 · CHECK · advisory lock

ERD · 정규화

27 테이블 6 축 · 파생 원장 분리

AUTH · REALTIME

JWT

엑세스 15 분 · 리프레시 회전 · 재사용 감지

OAuth 2.0 (카카오)

REST 2 회를 트랜잭션 밖에서

BCrypt

더미 해시로 타이밍 균등화

WebSocket · STOMP

수신 전용 · CONNECT 인증 · SEND 거부

SSE

1 회용 구독 티켓 · EventSource

INFRA · OPS

AWS EC2

t2.micro · systemd · 헬스체크 · 자동 롤백

nginx

단일 오리진 · TLS · WebSocket 프록시

GitHub Actions

러너 빌드 · scp 배포 · concurrency

certbot · duckdns

HTTPS · SameSite=Strict 쿠키

Supabase Storage

이미지 업로드 (트랜잭션 밖)

TOOLS · COLLAB

● Git · GitHub PR

GitHub PR 기반 협업

검증 과제

실제 DB 롤백 · 동시성 · 락 검증 보완 필요

React 19 · Vite

apiClient · useForm 공통화

AI 리뷰 활용

AI 리뷰 결과를 재현 · 실측 후 반영

문서화

api-spec 1,164 줄 · troubleshooting 7 건

세 프로젝트에서 권한 → 트랜잭션 → 실시간 · 배포까지 백엔드 책임을 확장했습니다

개인 기여는 Git 이력 · 코드 · 실측으로 확인되는 범위만 정리했습니다 . 이전 프로젝트의 한계가 다음 프로젝트의 설계 기준이 되었습니다 .

01 · 2026.05 · 2 인

Cooking Star

기본기 — 회원 · 인증 · 댓글 · 좋아요 · Kakao/Naver
검색 연동 · 관리자

- JSP · MyBatis · Security
- Git blame 6 개 도메인 100%

한계 : 자동화 테스트 부족 · 수동 검증 중심



02 · 2026.06-07 · 4 인

Haru Health

업무 정합성 — 결제 원자성 · PT 차감 원장 · 정산 배치
역등성 · SSE 알림 예외 처리

- 27 테이블 · 요구사항 117 건
- GitHub PR 기반 협업 · 실제 DB 검증 보완 필요

한계 : 미배포 · permitAll



03 · 2026.08-09 · 4 인 · EC2 운영 경험

댕댕댕

운영 설계 — 토큰 회전 · 재사용 감지 · REST+STOMP ·
AFTER_COMMIT · EC2 배포

- 커밋 110/219 · 엔드포인트 40
- PR 25 건 리뷰 · API 측정 조건은 6 장

한계 : CI 테스트 미포함 · 단일 인스턴스

프로젝트 간 개선 흐름

수동 JWT 검증 → 인증 필터 · 알림 예외 처리 → 커밋 후 발송 · 반환 코드 → ErrorCode · 로컬 실행 → 자동 배포

인증 → 반려동물 → 오픈채팅 → 배포까지, 서비스의 현관문과 복도를 맡았습니다

4 인 팀 · 반려동물 통합 케어 플랫폼 · 2026.08-09 AWS EC2 배포 · 운영, 현재 서버 종료. 담당 범위만 정리했습니다.

110 / 219

커밋 (병합 제외 · 본인 / 전체)

저장소 개설 · PR 25 건 리뷰 · 머지

40 개

REST 엔드포인트

member 13 · pet 7 · chat 20 (+ STOMP)

46 / 69

요구사항 담당

인증 12 · 인프라 7 · 펫 7 · 채팅 15 · 홈 5

1 회원 · 인증

BCrypt · JWT(15 분) · 리프레시 토큰 회전 · 재사용 감지 · 카카오 OAuth · 기기 관리 · 원격 로그아웃

2 오픈채팅

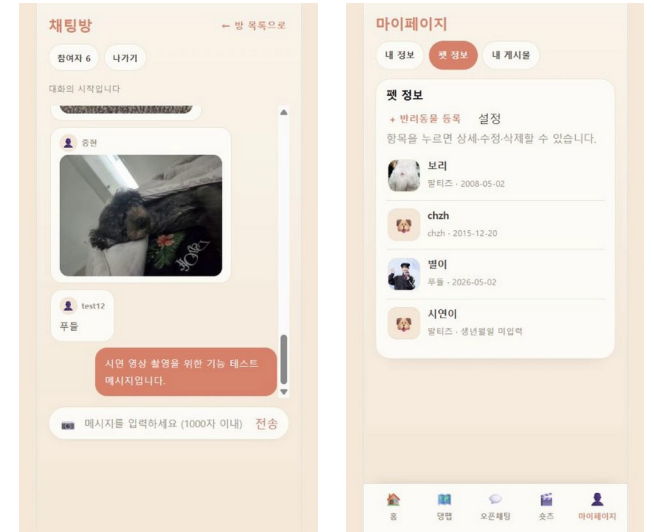
방 · 권한 3 종 · 강퇴 · 위임 · 발신 REST / 수신 STOMP · 읽음 · 공지 핀 · 이미지 메시지

3 반려동물 · 홈

소유자 격리 (전부 404) · @Version 낙관적 잠금 · 프로필 사진 Storage · 온보딩

4 공통 규약 · 배포

ApiResponse · 도메인별 ErrorCode · JWT 필터 · EC2 + nginx + GitHub Actions



오픈채팅

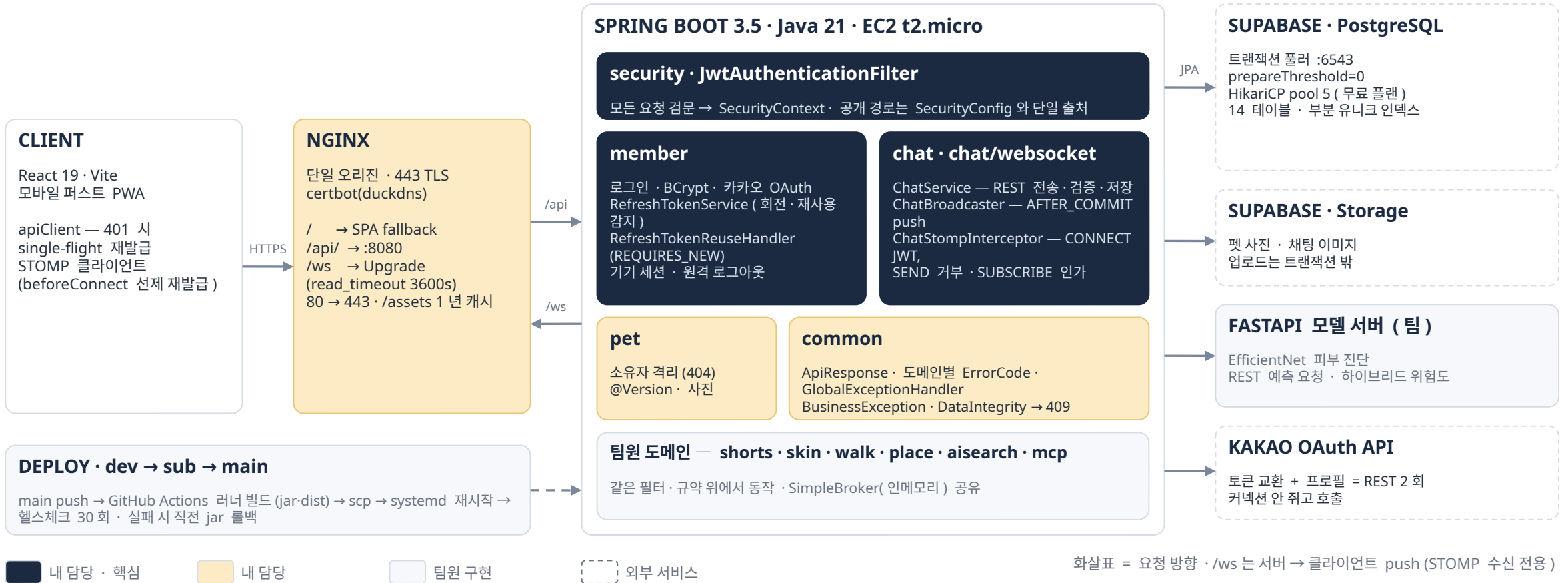
마이페이지 · 펫

실측

API 6 종 p95 45~78ms · 단일 사용자 배포 서버 · 각 20 회 · 2026-09-02

nginx 단일 오리진 뒤 3 개 서버 — Spring 이 허브 , FastAPI 는 모델 전용

제 담당은 Spring 의 member · chat · pet · common · security 패키지와 nginx · systemd · GitHub Actions 배포 경로 . 외부 I/O 는 트랜잭션 밖 , 방송은 커밋 후 .



재사용 감지가 '동작하는 것처럼 보이면서' 아무것도 하지 않았다

리프레시 토큰은 난수 · 해시 저장 · 회전, 재사용 시 전 기기 폐기. 그런데 폐기 UPDATE 직후 401 예외를 던지자 같은 트랜잭션이 롤백되며 폐기까지 되돌아갔습니다.

재현 : 재발급 (토큰 1→2) → 토큰 1 재발급 401 → 토큰 2 재발급 200 x

```
// BEFORE — 호출자 @Transactional 안
if (token.isRevoked()) {
    refreshTokenRepository.revokeAllByMemberId(...); // 폐기 UPDATE
    throw new BusinessException(INVALID_REFRESH_TOKEN);
} // RuntimeException → 롤백 → 위 UPDATE도 함께 사라짐
```

```
// AFTER — 별도 빈 + REQUIRES_NEW: 바깥이 롤백돼도 폐기는 커밋
@Component class RefreshTokenReuseHandler {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    void revokeAllOf(Long memberId) {
        repo.revokeAllByMemberId(memberId, now(), REUSE_DETECTED);
        repo.expireRotationGraceByMember(...); // 유예 안 토큰도 무력화
    }
}
```

수동 재현 결과 — 토큰 2 재발급도 401. 정상 회전 · 로그아웃 · 만료 경로도 확인. 폐기 로그만으로 DB 커밋을 판정하지 않습니다.

검토한 대안

(선택) REQUIRES_NEW 별도 빈

프록시를 거치도록 별도 빈으로 분리. 추가 커넥션이 필요하므로 풀 5 개에서 동시 호출 · 잠금 대기 검증 필요

noRollbackFor = BusinessException

한 줄이지만 다른 실패 경로의 롤백까지 사라짐 — 탈퇴 회원 재발급 시 회전 되돌리기가 깨짐

반환값으로 올려 호출자가 던지기

호출자도 @Transactional — 같은 트랜잭션이라 문제가 그대로

유예 30 초 : 응답 전 재시도 · 동시 탭의 옛 토큰 제출 오탐 완화. 경계 동시 요청은 추가 검증 · 근거 : docs/troubleshooting.md 4 번

보낼 땐 REST, 받을 땐 WebSocket — DB 에 확정된 것만 방송하고, 느린 작업은 트랜잭션 밖으로

온라인 수신을 커밋 후 push 로 전환했습니다. afterId 재조회도 제공하지만, 커밋·ID 순서 역전과 서버 중단 시 누락·중복은 추가 검증 과제입니다.



```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void onMessageCreated(ChatMessageCreatedEvent e) {
    safely("메시지 푸시", () ->
        messagingTemplate.convertAndSend(roomTopic(e.roomId()),
            ChatEvent.message(e.message())));
}
// ChatService 는 WebSocket 을 import 하지 않는다 - 의존은 websocket -> chat 한 방향
// 롤백된 강퇴로 멀쩡한 사용자의 연결을 끊지 않기 위해 강퇴 세션 종료도 같은 경로
```

```
// ChatService.join - 의도적으로 비트랜잭션 ( 풀 5 개 )
try { joinedId = memberRepo.save(ChatRoomMember.join(...)).getId(); }
catch (DataIntegrityViolationException e) {
    if (!existsActive(roomId, memberId)) throw e; } // 중복 입장만 흡수
revertIfJoinLost(roomId, memberId, joinedId); // 강퇴·정원초과면 되돌림
```

SEND 는 인터셉터에서 거부 · 인증은 CONNECT 프레임에서

발신은 REST 로 통일하고 클라이언트 SEND 를 인터셉터에서 거부합니다. CONNECT 헤더의 JWT 와 SUBSCRIBE 참여 권한을 검사합니다.

커넥션 풀 5 개가 설계를 지배했다

- UNIQUE 충돌 처리 후 상태 재확인·보상. 보상 전 서버 중단과 동시 정원 초과는 추가 검증 필요
- 업로드·카카오 REST·BCrypt 는 DB 쓰기와 분리. 풀 대기 개선량은 미측정

공통 규약 · EC2 자동 배포 · 재현 시나리오를 통한 리뷰 판단

팀원 4 명이 같은 모양으로 응답하고 같은 방식으로 실패하게 만들고, PR 리뷰에서 지켰습니다. AI 리뷰 결과는 재현 · 실측 후 반영했습니다.

25 PR

리뷰 · 머지 담당

dev → sub → main 브랜치 전략

재현 · 실측

AI 리뷰 결과 검증

재현 · 코드 확인 후 반영

7 건

트러블슈팅 문서

증상 → 원인 → 대안 → 변경 → 결과

p95 ≤ 78ms

담당 조회 API 6 종

단일 사용자 · 각 20 회 · 09-02

1 공통 규약

- ApiResponse 봉투 · 도메인별 ErrorCode(전역 유일 규칙) · GlobalExceptionHandler
- JwtAuthenticationFilter — Haru 의 ' 전역 permitAll + 수동 검증 ' 한계 개선
- 존재 여부를 노출하지 않는다 — 실패 사유 통일 · 타 인 / 삭제 / 없음 전부 404

2 배포 · 인프라

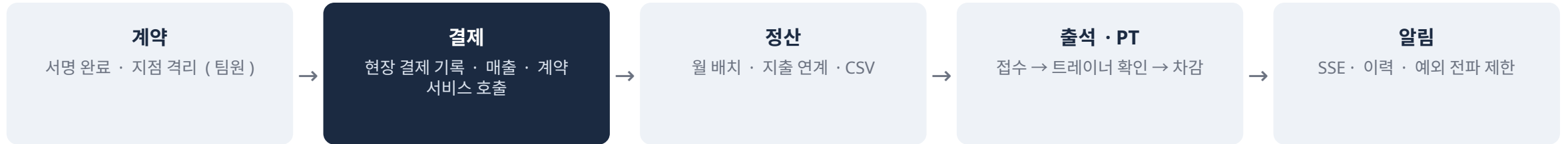
- 프리티어 RAM 1GB → 러너에서 jar-dist 빌드, scp → systemd → 헬스체크, 실패 시 직전 jar 롤백
- nginx 단일 오리진 — CORS 제거, 리프레시 쿠키 SameSite=Strict, /ws 장기 연결 프록시
- 한계 : 파이프라인에 테스트 미포함 (팀 공용 DB 부작용)

3 리뷰를 검증하는 방식

- P1 · 원격 로그아웃 — 잠금 뒤 DEVICE_REVOKED 재확인
- P1 · 회전 유예 — 누락된 폐기 UPDATE 추가
- 프로브 : Spring 6.2 의 해당 AFTER_COMMIT 예외 후 201 유지 관찰 . 다른 리스너 방식에는 일반화하지 않음

계약 → 현장 결제 기록 → 정산 → 출석 흐름

4 인 팀 · 요구사항 117 건 · 테이블 27 개 · 미배포 (로컬 검증). payment · settle · item · checkInout · alarm 의 Service·Mapper 계층을 담당했습니다 .



1 멀티테넌트 데이터 격리

화면이 보낸 지점 번호 대신 JWT 의 username 으로 회원을 재조회해 gym_id 를 강제 (WHERE gym_id = (SELECT ... WHERE username = #{username}))

2 핵심 업무는 한 트랜잭션으로

현장 결제 결과 · 매출 기록과 계약 활성화 서비스 호출 , 출석 확인 · PT 차감에 트랜잭션 경계 적용 . 실제 DB 롤백 · 중복 처리는 추가 검증 과제

3 남의 도메인은 읽고 내 원장에 기록

계약 도메인은 팀원 담당 . 결제 흐름에서 계약 조회 · 활성화 서비스를 호출하고 , PT 사용량은 h_pt_manage 에 기록

4 알림 예외 전파 제한

sendAlarmSafely 로 호출부 예외 전파 제한 . DB 오류의 롤백 영향은 미검증이며 , 후속 댕댕에서는 AFTER_COMMIT 발송 적용

정산 · 매출

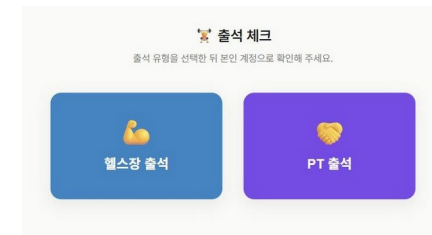
정산 매출

비율과 지출 내역을 확인하고 기간별 정산 업무를 관리합니다.

기간: 2026.06.01 ~ 2026.06.07

일련번호	회원 ID	회원명	결제 금액	결제 일자	결제 방법	잔액	비고
59	77771234	PT - 대한 헬스 회원1234	₩0	2026.06.01	PT회원 (₩30,000)	₩26,539,109	잔액 3,000원 지출 400원
58	77771234	이동권 - 대한 헬스 회원1234	₩500,000	2026.06.01	미사용	₩3,060,044	잔액 3,000원 지출 400원
57	77770126	PT - 대한 헬스 회원27	₩150	2026.06.01	미사용	₩3,060,044	잔액 3,000원 지출 400원
56	77770121	PT - 77770121	₩30,000,000	2026.06.01	미사용	₩3,060,044	잔액 3,000원 지출 400원
33	77770128	PT - 대한 헬스 회원29	₩130	2026.06.01	미사용	₩3,060,044	잔액 3,000원 지출 400원
32	77770129	PT - 대한 헬스 회원30	₩149	2026.06.01	미사용	₩3,060,044	잔액 3,000원 지출 400원

정산 · 지금 처리 (담당)



출석 키오스크 (담당)
화면 : 로컬 시드 데이터

React → Spring(MyBatis) → PostgreSQL — 내 담당은 현장 결제 기록 · 정산 · 출석 · 알림 서비스 계층

단일 DB · 단일 스키마 row-level 멀티테넌트 · 화면이 보낸 지점 번호는 믿지 않고 JWT 의 username 으로 회원을 재조회해 gym_id 를 강제합니다 .



현장 결제 기록은 한 트랜잭션으로, PT 차감은 조건부 UPDATE 로 판정

현장 결제 결과를 기록하는 사람은 회원이 아니라 사장님입니다. PT 차감은 DB 의 원자적 UPDATE 에 맡겼습니다.

PayService.checkout — 요약 의사코드

```
@Transactional
checkout(dataId, owner, couponId) {
    c = findPayableContract(dataId, owner); // ① sender = 로그인 사장님
    amount = applyCoupon(c.receiverId, ...); // ② 소유자 · 미사용 · 만료 · 지점 · 카테고리
    if (amount == 0) installment = 0; // ③ 0 원이면 할부 강제 해제
    insertPay(c, amount); // ④ h_pay
    markCouponUsed(couponId); // ⑤ h_payment (매출)
    insertRevenue(c, amount); // ⑥ 실패 시 결제 취소
    if (!activateContract(c)) throw ...;
}
```

```
-- 잔여 = contract.quantity - pt.used_count (파생, 별도 저장 안 함)
UPDATE h_pt_manage
SET used_count = used_count + 1,
    completed = (used_count + 1 >= total_count)
WHERE data_id = :dataId AND used_count < total_count;
-- 영향 행 0 → 예외 → 롤백. 원장 생성은 ON CONFLICT (data_id) DO NOTHING
```

행위자와 기록 대상의 분리

- 처음엔 receiver_id 를 로그인 사용자로 검증했지만 실제 흐름은 카운터에서 사장님이 현장 결제 결과를 기록합니다. sender 와 기록 대상 회원을 분리했습니다.
- 외부 PG 미연동 · 계약 도메인은 팀원 담당, 결제 흐름에서 조회 · 활성화 서비스 호출 연동

월 정산 배치 역등성

pg_advisory_xact_lock(gymId) 로 지점별 키 잠금 후 존재 확인 → 0 건일 때만 INSERT.
지급 처리는 '미지급' 일 때만 갱신, 금액은 요청 body 를 버리고 DB 값으로

알고 있는 한계

동시 결제 창 존재: 복수 결제 규칙 정의 후 제약 결정. 동일 출석의 중복 차감 · 하루 1 회 출석 · DB 롤백 · 락은 추가 검증 과제

row fan-out 으로 깨진 페이지네이션을 LATERAL 로 고치고 , SSE 구독은 1 회용 티켓으로

MyBatis 를 고른 이유는 집계 · 조인 · 동적 정렬이 많은 화면이었기 때문입니다 . PostgreSQL 전용 기능을 그대로 썼습니다 .

매출 목록 — 요약 (실제 : payment 매퍼 XML)

```
-- BEFORE: 같은 계약에 결제 2건 → 매출 1건이 2행으로 . 건수 쿼리와 어긋나 페이지 깨짐
JOIN h_pay p ON p.data_id = pm.data_id
-- AFTER: 바깥 행마다 최신 결제 1건만
LEFT JOIN LATERAL (SELECT pay_method FROM h_pay
                    WHERE data_id = pm.data_id
                    ORDER BY p_id DESC LIMIT 1) p ON true
-- 목록 · 건수 · CSV 세 쿼리의 WHERE 는 <sql> + <include>로 단일화
```

SSE 구독 보안 · 무인증 키오스크 방어

- EventSource 는 Authorization 헤더를 못 보냄 → Bearer 로 1 회용 티켓 (SecureRandom 32B · TTL 60s · 조회 즉시 remove) 발급 후 구독
- 키오스크 레이트 리미터 — 계정 축 (5 회 → 1 분) + IP 축 (20 회 → 5 분) , password spraying 대비 두 키

● 확인한 것

- 코드 : 타 지점 접근 거부 · 정산 분기
- 코드 : 현장 결제 기록 트랜잭션 경계
- 코드 : PT 조건부 UPDATE
- 코드 : 알림 catch · SSE 티켓
- 실제 DB 기반 롤백 잠금 동시성 시나리오 검증 미완료

● 한계 → 다음

- 미배포 · 운영 환경 성능 측정 없음
- 후속 적용 : JWT 필터 · 리프레시 회전
- 보완 : DB 롤백 · 알림 DB 실패
- 보완 : PT 중복 차감 · 정산 잠금
- 보완 : 결제 규칙 · 유니크 제약 · 타임존

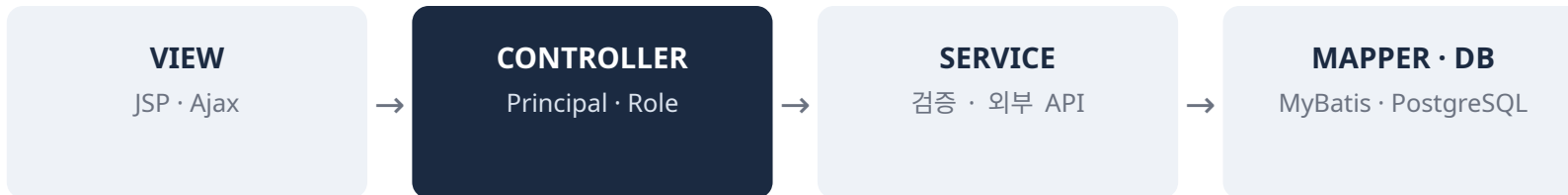
Spring MVC 위에 인증 정보와 외부 API 응답이 들어오는 위치 — 첫 프로젝트의 계층 구조

2 인 팀 · 회원 · 인증 , 댓글 · 좋아요 · 북마크 · 팔로우 , Kakao Local·Naver Blog, 관리자 · 방문 집계 담당 . 요리 기록 게시판 공동 구현 .



첫 Java/Spring 팀 프로젝트 — 기본기와 후속 개선 과제

2 인 팀 · 회원 · 인증 , 댓글 · 좋아요 · 북마크 · 팔로우 , Kakao Local·Naver Blog, 관리자 · 방문 집계 담당 . 요리 기록 게시판 공동 구현 .



서버 기준 권한

BCrypt · Form Login · 역할별 URL
· 댓글 삭제는 SQL 에도 작성자 조건

데이터 무결성

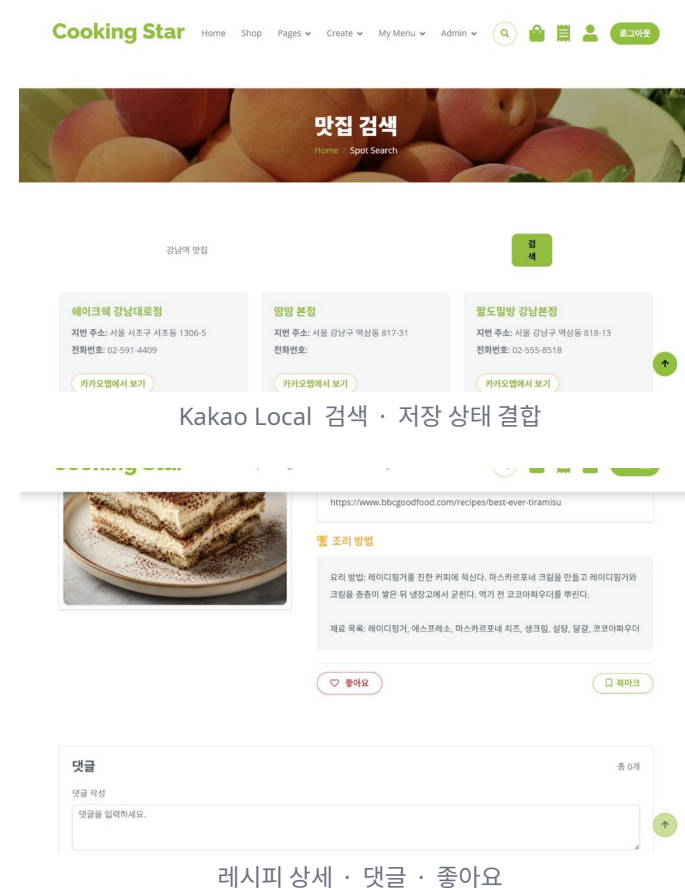
좋아요 음수 하한 · 방문 ON
CONFLICT
제약 · 집계 일치 · 동시 요청은 추가
검증

외부 · 내부 결합

Kakao Local·Naver Blog 응답을
DTO 로 변환하고 저장 상태 결합

첫 프로젝트의 한계 → 다음 프로젝트의 개선 기준

- 검증 : 실제 DB 롤백 · 동시성 · CI 보완 필요
- 인증 : 세션 → JWT 필터 · 회전 경험 . CSRF 방어는 별도 과제
- 외부 API: 후속 I/O·DB 경계 분리 . 타임아웃 · 재시도 정책은 별도 과제



세 프로젝트에서 확장한 책임 — 검증 범위와 다음 과제

구현과 수동 확인의 범위를 구분하고, 실제 DB 검증과 CI 를 현재 보완 과제로 정했습니다.

● 세 프로젝트에서 확인한 것

- 권한 : Principal·SQL 작성자 조건·회원 조회로 gym_id 적용
- 트랜잭션 : 결제 경계·알림 catch·토큰 폐기 분리
- 실시간 : 커밋 후 방송·외부 I/O 와 DB 쓰기 분리
- 배포 : EC2 자동 배포·HTTPS
- 측정 : API 6 종 p95 45~78ms·각 20 회·단일 사용자
- 협업 : PR 25 건 리뷰·공통 규약·트러블슈팅 7 건

● 현재 한계

- CI·부하 테스트 미실시
- 핵심 인증·트랜잭션 시나리오 실제 DB 검증 필요
- 보상 중단·메시지 복구 추가 검증 필요
- SimpleBroker 인메모리·단일 인스턴스
- Haru 미배포·동시 결제 창·타임존 과제

● 현재 보완 과제

- 핵심 인증·입장 보상·위임 경쟁의 실제 DB 검증과 CI 분리
- Haru 롤백·알림 DB 실패·PT 중복·정산 잠금 검증
- 결제 이력 규칙·제약·타임존·인증 필터 보완
- 부하·복구 측정 후 WebSocket 다중 인스턴스 확장 검토

● THANK YOU

세 프로젝트에서 배운 기준으로, 안전한 업무 흐름을 끝까지 책임지겠습니다

1

권한과 입력은 서버가 판단한다

Cooking Star · Principal · Role · SQL 소유자 조건

2

함께 성공할 것과 실패해도 되는 것을 나눈다

Haru Health · 결제 원자성 · PT 원장 · 알림 예외 전파 제한

3

제약을 설계에 반영하고 커밋 이후를 책임진다

땡땡땡 · 토큰 회전 · AFTER_COMMIT · 풀 5 개 · 배포

송태현 · Backend Developer

uiop7000@naver.com · github.com/Taehyun-0502